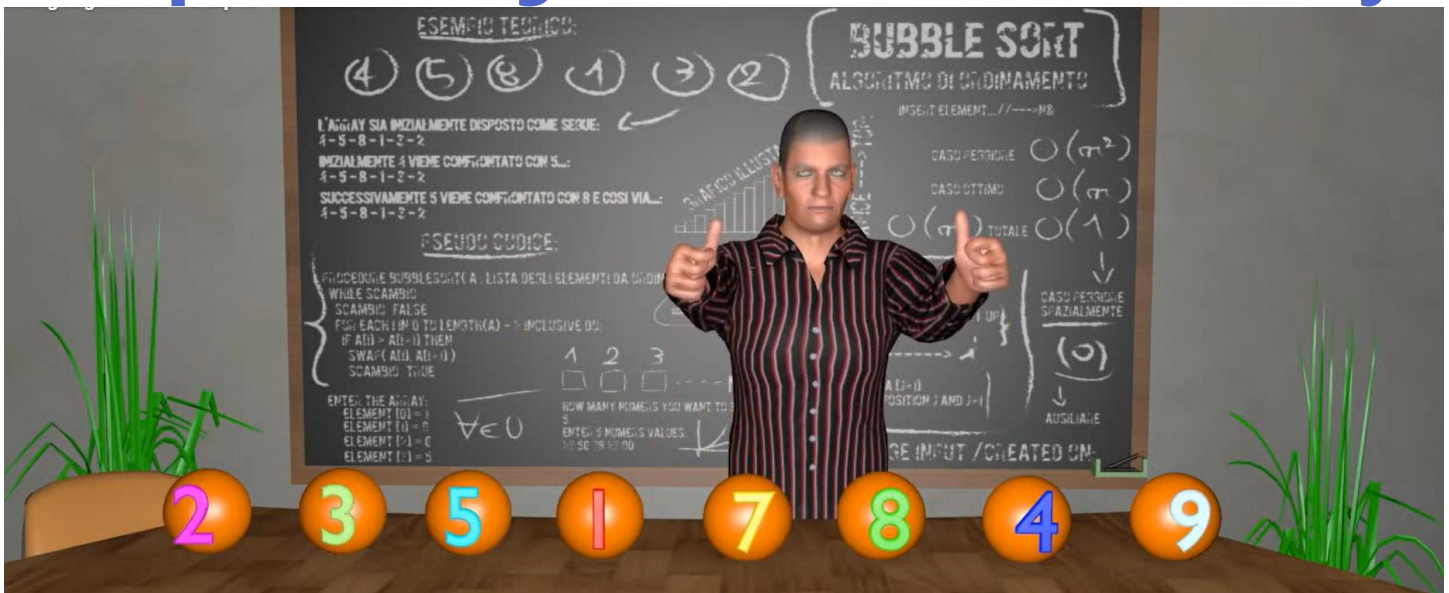




## LAB 4, FALL 2026

# Simple Sort Algorithms & Time Efficiency



## CONCEPTS FOR LAB 4

- Bubble Sort
- Selection Sort
- Insertion Sort
- Scrambling Arrays (randomizing order)
- Dynamic memory and arrays
- Reading from text files
- Using the chrono library for timing algorithms

## THIS IS NOT A PAIRED PROGRAMMING ASSIGNMENT

All students are to complete this assignment on their own. You all can talk to each other but you definitely shouldn't have the same code! If you do, that is considered cheating for this assignment (refer to syllabus academic integrity policy).

## LAB 4 POLICIES

### Generative AI Policy for Lab 4

You are **NOT** allowed to use GitHub Copilot, OpenAI Codex, or any other AI generation tool for **code generation**. Therefore, if you currently have GenAI enabled within your IDE, you need to **disable it for completing this lab**. You should try to implement the sorting algorithms yourself without outside help so that you can learn these algorithms and how to implement them.

You may use Generative AI to aid in [troubleshooting your code](#). **Do not upload the assignment directions and ask it to generate the solution or parts of the solution for you.**

**Uploading the assignment to generate code or enabling code generation/completion within VSCode will result in an academic integrity violation charge.**

## WHAT YOU CAN'T DO/USE IN THIS LAB

**If you do any of the following, you will receive a 0 on some or all of the assignment and possibly get charged with an academic integrity violation charge (this is very serious).**

1. You may not use more advanced code or syntax in your lab assignment **that has not been taught** in CSC 1310 class **without referencing your source**. How do you reference your source? By providing a link/URL in a multi-line comment in the section of code where you use the advanced code. **Otherwise, this will be considered plagiarism.** You should be able to explain every line of code that is in your submission.
2. You may not (ever) use the following in your programming & lab assignments in CSC 1310:
  - while(true) or while(1)
  - goto statements
  - recursively calling main
  - exit()
  - ternary operators (any syntax requiring a '?')
  - range-based for loops
  - try-catch
  - getline(cin>>ws, variable);
  - lambda functions
  - defining all programmer-defined functions above the main function
  - code that has no comment block at the top indicating the filename, author, date, and purpose
3. You **may not use generative AI (genAI)** to generate code, but you can use it to help in troubleshooting problems as described in the Generative AI Policy above.
4. You may not share your code with anyone who is NOT your paired programming partner (see next page).

## GIVEN FILES

You will be given part of the program and several different .txt files to test your program.

- **lab4.h**  
**lab4.h** is completely written for you and you shouldn't have to change anything.
  - Make sure you read through it, though, as it talks about **namespace std** and the more recommended way to handle it when programs are more complex.
  - You will also want to use this so you know what your **function headers** should look like.
- **lab4.cpp**  
You will have to add code to this source file to make it work. More detail on what to add in section in this assignment on **"functions to add"**
- Text files to test your code
  - **movies\_short.txt** – use this one until you know your code works and your sort algorithms are working correctly. Only implement one sort function at a time and then test it (INCREMENTAL PROGRAMMING). Just comment out the functions you haven't implemented yet in your header file.
  - **movies.txt** – use this one to time how long it takes to run your sort algorithms
  - **movies\_halloweenHorror.txt** – I included this one just for fun. You don't have to use it

- **movies\_very\_long.txt** – this file has 233547 movie titles in it. It will take at least ~10.5 minutes just to run bubble sort on an array created with this file. So only use this one if you are bored and curious how long it will take to run these different sort algorithms on a huge array.

**Reference/Credit for the .txt files:** I used ChatGPT 5 to create the short list and part of the Halloween horror list. The majority of movies.txt and movies\_very\_long.txt were created by exporting data from IMDB <https://datasets.imdbws.com/> and then downloading title.basics.tsv.gz, unzipping it, opening it in Excel to format, copying the title column into a .txt file and then saving it.

## DON'T FORGET TO FILL OUT THE LAB REPORT

**After** you have completed this assignment, fill out the lab report and submit your lab report in your zip file along with your source files. Name your screenshot **lab4ReportProof**.

**Lab Report Link:** [https://tntech.co1.qualtrics.com/jfe/form/SV\\_0iJcCTCBgguPDpQ](https://tntech.co1.qualtrics.com/jfe/form/SV_0iJcCTCBgguPDpQ)

Create a **zip** file named **username\_lab4.zip** containing the following files and upload it to Canvas in the **Lab 4** assignment. Replace username with your TTU. Example: **acrockett43\_lab4.zip**

- **lab4.h**
- **lab4.cpp**
- **movies.txt**
- **movies\_short.txt**
- **Lab4\_Documentation.pdf**
- **Lab4ReportProof**

## CHRONO LIBRARY – HOW TO TIME ALGORITHMS!

In this lab and in your Program 2 assignment, you will time how long it takes your sorting algorithms to run.

The C++ **chrono** library provides **accurate** high-resolution timers.

When you call `auto t0 = steady_clock::now();` you capture the current time from the system clock, which will represent your algorithm's **start time**. The **system clock** is a built-in electronic oscillator inside a computer that keeps track of the passage of time. It sends out a regular sequence of electrical pulses that synchronize all the components. Every action in the CPU (fetch, decode, execute) happens in step with clock pulses. The rate of pulses (the *clock speed*) is measured in hertz (Hz), usually gigahertz (GHz) for modern CPUs. The `now()` function returns a **time\_point** object, which represents the current tick count of the steady clock. The steady clock uses the system's high-resolution hardware timer. Each pulse (tick of the oscillator) increments a hardware counter. So, `steady_clock::now()` queries that counter and is the number of elapsed pulses (converted into nanoseconds, microseconds, or another duration unit) since the steady clock's epoch. Epoch is just some fixed point in the past chosen by the implementation (often the time when the system booted or when the clock source was initialized).

Later, after your algorithm runs, you call `auto t1 = steady_clock::now();` to capture the **end time**.

Subtracting these two gives you the elapsed time duration.

You can then use `duration_cast<milliseconds>(t1 - t0).count()` to convert this duration into **milliseconds**. This way you can compare the efficiency of different algorithms.

## FUNCTIONS TO ADD

You will need to implement the following functions listed below in **lab4.cpp**

You can use the example code provided for you to help you with the sort algorithms but you will have to change them to handle sorting strings instead of integers.

- **bubbleSort()** – should implement the efficient version of the bubble sort algorithm
- **printArray()** – should print the array elements (look at sample output)
- **selectionSort()** – should implement the selection sort algorithm
- **insertionSort()** – should implement the insertion sort algorithm
- **doubleArray()** – should double the size of the given array – notice this function is called from the **readFromFile()** function, which is **already written for you**.
  - Parameters
    - a reference to the array pointer to resize
    - **oldMaxSize** – current maximum size of the array
  - Should return the new maximum size of the array (which should be the old max size multiplied by 2)
  - Pseudocode:
    - Create a variable to hold the new maximum size.
    - If the **oldMaxSize** is greater than zero, then do the following:
      - Set **newMaxSize** to **oldMaxSize \* 2**
      - Print out that the array size was doubled from [old size] to [new size]
      - Dynamically allocate a string array of the new size
      - Copy all the elements from the old array to the new array
      - Delete the old array
      - Assign the new array's address to the pointer that held the old array (this way it gets sent back to main)
    - Return **newMaxSize**

## DOCUMENTATION DOCUMENT

1. Create a document named **Lab4\_Documentation.docx**.
2. On the first line of this document, type your first and last name.
3. On the second line, type **Lab 4 Output with movies.txt**.
4. Then, run your program using **Sample Output 2 (using movies.txt)** from the **Lab 4 Sample Output\_CSC 1310\_S26.pdf** document.
5. Copy your output from your terminal or command prompt and paste it into Lab4\_Documentation.docx.
6. Select all the text in the document and change the Line Space Options in the document to be single spaced (1.0) and remove any space before or after each line.
7. Save this file as a PDF so that it is named **Lab4\_Documentation.pdf**.

## SAMPLE OUTPUT

Look at the sample output in the separate document named **Lab 4 Sample Output\_CSC 1310\_F26.pdf**.